

Public Service Reporter — Technical Documentation

Backend

The repository contains the logical and authorization part of the **public-service-reporter** project, a platform created to facilitate communication between residents and municipal offices. It automates the process of detecting, reporting, and monitoring problems in public spaces.

1. Technology Stack (Tech Stack)

- **Main Framework:** Django 5.x / Django REST Framework (DRF)
- **Authentication:** TokenAuthentication (DRF Token Scheme)
- **Database:** Relational (mapped via Django ORM)
- **Access Control:** Role-Based Access Control (RBAC) with custom permission classes

2. Architecture and Directory Structure

The backend project is organized in a modular fashion, isolating authentication logic from business modules.

```
backend/  
├── apps/  
│   ├── service_auth/      # Authentication and user module (label: service_auth)  
│   │   ├── apps.py        # Django application configuration (BigAutoField, label)  
│   │   ├── models.py      # Extended user model (AbstractUser)  
│   │   ├── serializers.py  # Data serializers and input role validation  
│   │   ├── urls.py        # URL path mapping for the auth module  
│   │   └── views.py        # API view classes (RegisterView, LoginView)  
│   └── reports/          # Citizen reports module  
│       └── permissions.py  # Object-level and global permission definitions
```

3. Role-Based Access Control (RBAC)

The application implements a three-tier user role division, defined directly in the database model and validated at the data presentation layer (serializers):

- **standard Resident:** The basic role assigned by default. It has permissions to create reports and browse public issues.
- **worker Municipal Employee / Official:** A role with elevated privileges. Responsible for the technical and administrative handling of reports within a given region.
- **admin System Administrator:** Full administrative access to manage the database structure, users, and global configuration.

4. Data Models

User Model (Extends AbstractUser)

A custom user model integrating native Django security mechanisms with the business logic of the role system.

Field	Data Type	Properties	Description
<code>id</code>	Integer	Primary Key, Auto Increment	Unique user identifier.
<code>username</code>	CharField	Unique, Max length: 150	Username used during login.
<code>password</code>	CharField	Write Only (Hash)	Secure password hash generated by <code>set_password()</code> .
<code>email</code>	EmailField	Optional	User's email address.
<code>role</code>	CharField	Choices, Max length: 10	User role in the system: <code>standard</code> , <code>worker</code> , <code>admin</code> . Default: <code>standard</code> .
<code>data</code>	TextField	Nullable, Blank	Text field intended for additional regional metadata (e.g., name of the assigned municipality).

5. Security and Permissions Layer

Located in the `apps.reports.permissions` module, this layer controls access to resources at the view and individual object levels:

- **IsOwner (Object-level permission):** Verifies the ownership relation of the object. Returns `True` exclusively when the user field in the business entity matches the authenticated user in the request (`obj.user == request.user`). This prevents unauthorized modification of reports by other residents.
- **IsAdmin (Global permission):** Allows access to a protected endpoint only when the requesting user has successfully passed the authentication process and their role field is set to `admin` or `worker`.

6. API Documentation (Endpoints)

All endpoints return responses in JSON format.

Method	Endpoint	Operation Description	Required Permissions
POST	<code>/api/auth/register/</code>	Registration of a new account and role validation.	<code>AllowAny</code>
POST	<code>/api/auth/login/</code>	Authentication of data and issuance of a DRF token.	<code>AllowAny</code>

Request/Response Structure Specification

POST /api/auth/register/

Accepts structural data of a new user. Encrypts the password before saving the entity to the database.

Example Request Body:

```
{
  "username": "jan_kowalski",
  "password": "SuperSecurePassword123",
  "email": "jan.kowalski@example.com",
  "role": "standard",
  "data": "Gmina Solina"
}
```

Example Response (201 Created):

```
{
  "message": "User has been successfully registered (MOCK)",
  "user": {
    "username": "jan_kowalski",
    "role": "standard",
    "data": "Gmina Solina"
  }
}
```

POST /api/auth/login/

Verifies credentials against the database using the `authenticate()` method.

Example Request Body:

```
{
  "username": "jan_kowalski",
  "password": "SuperSecurePassword123"
}
```

Example Response (200 OK):

```
{
  "token": "a1b2c3d4e5f6g7h8i9j0klmnopqrstuvwxyz1234",
  "role": "standard",
  "user_id": 12
}
```

7. Work Status and Milestones (Backend TODO)

1. **Registration Refactoring:** Removal of the mock structure `(MOCK)` from `RegisterView` and replacing it with full serialization of the created database object.

2. **Verification Module:** Implementation of an email address confirmation mechanism using activation tokens sent via email.
3. **Integration with Reports:** Linking `IsOwner` and `IsAdmin` permissions with CRUD views for reports (reports module).